Centro Universitário Católica de Quixadá

API RESTful para um Sistema de Gestão de Eventos

Avaliação: Esta atividade vale 1 ponto na composição da nota da AP1.

Objetivo: Unificar os conceitos estudados na construção do back-end de um Sistema de Gestão de Eventos (Event Manager), utilizando Node.js, Express e Sequelize.

Instruções Iniciais

Olá! Hoje vamos colocar a mão na massa e construir a fundação da nossa aplicação web. Seu objetivo é desenvolver uma API RESTful do zero, passando pela configuração do servidor, criação das rotas, validação dos dados de entrada e persistência em banco de dados utilizando o ORM Sequelize. Construiremos a base para um Sistema de Gestão de Eventos, onde será possível criar eventos com limite de vagas e controlar as inscrições. Lembre-se de seguir o passo a passo com atenção, pois esta atividade valerá 1 ponto na AP1.

Passo 1: Estrutura Base e Express

1. Crie uma pasta para o projeto e inicialize o Node.js com o comando:
   npm init -y
2. Instale o framework Express, que simplifica a criação do nosso servidor:
   npm install express
3. Crie um arquivo chamado 'server.js' e importe o Express, instanciando a aplicação e abrindo uma porta de escuta (ex: 3000).
4. Crie um arquivo para as rotas (ex: eventRoutes.js) utilizando o express.Router(). Defina as seguintes rotas básicas, que por enquanto retornarão apenas mensagens JSON:
   - POST /events (Para criar um novo evento)
   - GET /events (Para listar os eventos disponíveis)
   - POST /events/:id/register (Para registrar um participante em um evento)

Passo 2: Validação de Dados com Joi

Vamos garantir que apenas dados consistentes cheguem ao nosso banco. Desta vez utilizaremos o Joi para a validação.

1. Instale a biblioteca Joi:

npm install joi
2. Crie um middleware de validação para a rota POST /events que exija o seguinte formato no body:
- title: string, obrigatório
- date: data (ISO), obrigatório
- capacity: número inteiro, obrigatório e maior que zero (limite de vagas)
3. Crie outro middleware para a rota POST /events/:id/register, exigindo:
- participantName: string, obrigatório
- email: string, formato de email válido, obrigatório
4. Se a validação falhar em qualquer um dos casos, intercepte a requisição retornando status 400 (Bad Request) com os detalhes do erro.

Passo 3: Configuração do ORM Sequelize e Banco de Dados

Utilizaremos o Sequelize, um dos ORMs mais robustos para Node.js, conectado ao SQLite para facilitar o ambiente local.
1. Instale o Sequelize e o driver do SQLite:
npm install sequelize sqlite3
2. Crie um arquivo 'database.js' para configurar a conexão com o banco de dados:
const { Sequelize } = require('sequelize');
const sequelize = new Sequelize({ dialect: 'sqlite', storage: './database.sqlite' });
module.exports = sequelize;
3. Defina os modelos de dados (Models). Crie um arquivo 'Event.js' e defina os campos title, date e capacity.
4. Crie um arquivo 'Registration.js' definindo os campos participantName, email e eventId (relacionamento com Event).
5. Estabeleça o relacionamento entre os modelos: Um 'Event' tem várias 'Registrations' (hasMany) e uma 'Registration' pertence a um 'Event' (belongsTo).
6. No 'server.js', faça a sincronização dos modelos com o banco utilizando sequelize.sync().

Passo 4: Implementando a Regra de Negócio nas Rotas

Agora vamos integrar o banco de dados e aplicar a lógica principal da nossa aplicação.
1. Rota POST /events: Utilize o modelo Event para criar um novo registro no banco com os dados validados.
2. Rota GET /events: Retorne todos os eventos. Desafio: inclua a contagem de inscrições (Registrations) atuais para cada evento na resposta.
3. Rota POST /events/:id/register: Esta é a rota mais complexa.
a. Busque o evento pelo ID informado na URL.
b. Se o evento não existir, retorne 404.
c. Conte quantas inscrições já existem para este evento.
d. Se o número de inscrições for igual ou maior que a capacidade ('capacity') do evento, retorne status 403 informando que o evento está lotado.
e. Se houver vaga, crie o registro na tabela Registration e retorne status 201.

Passo 5: Testes Manuais

Utilize um cliente HTTP (como o Thunder Client, Postman ou Insomnia) para enviar requisições à sua API e garantir que todas as regras funcionam:

1. Tente criar um evento sem capacidade definida (deve ser barrado pelo Joi).
2. Crie um evento válido (ex: capacidade 2).
3. Registre 2 participantes neste evento (as inscrições devem ocorrer com sucesso).
4. Tente registrar um 3º participante (a API deve barrar e informar que está lotado).
5. Liste os eventos e verifique se as informações estão corretas.

Checklist de Entrega

- Antes de submeter o seu trabalho, verifique se você cumpriu todos os requisitos abaixo. Marque mentalmente (ou imprima e marque) cada item concluído:
- O servidor Node.js com Express está rodando sem erros (ex: na porta 3000).
- As rotas de criação (POST /events) e listagem (GET /events) foram implementadas e isoladas no Router.
- A rota de inscrição (POST /events/:id/register) foi criada com sucesso.
- A validação de dados com Joi está funcionando e barrando requisições incorretas (status 400).
- O Sequelize foi configurado corretamente em conjunto com o SQLite.
- Os Modelos 'Event' e 'Registration' foram criados e o relacionamento (1:N) foi estabelecido.
- A regra de negócio de lotação máxima (capacity) está funcionando (barrando inscrições extras com status 403).
- Todos os testes no cliente HTTP (Thunder Client, Insomnia ou Postman) foram aprovados.
- O código fonte está devidamente organizado e foi commitado no repositório do GitHub para o envio.